

Day 03: USER MANUAL - 2

Secure Coding Practices

*Implementing SSL / Certificate Pinning on Android with
Network Security Configuration*

Table of Contents

1 . Introduction.....	3
2. Certificate Pinning implementation guide.....	4
2.1 — Capture the server's certificate chain.....	4
2.2 — Compute the SHA-256 SPKI Pin.....	4
2.3 — Create the Network Security Configuration XML.....	4
2.4 —Reference the Configuration in AndroidManifest.xml.....	5
2.5 — Write Standard Networking Code.....	6
2.6 —Build, Install, and Verify Normal Traffic.....	6
2.7 — Verification : Prove Pin Enforcement.....	6

1 . Introduction

As mobile applications handle sensitive data, securing network traffic against man-in-the-middle (MITM) attacks and rogue Certificate Authorities is crucial. This guide covers implementing SSL Certificate Pinning using Android's declarative Network Security Configuration.

Where Things Live in Android Studio :

File or Directory Path	Purpose / Contents in This Project
app/src/main/AndroidManifest.xml	Declares internet permissions and links the network security config file.
app/src/main/res/xml/ network_security_config.xml	Houses domain-specific rules and SHA-256 SPKI pin-sets.
app/src/main/java/.../MainActivity.kt	Standard OkHttp network requests with zero custom pinning code.
app/build.gradle (Module: app)	Manages dependencies like OkHttp and Coroutines.
Logcat panel (bottom toolbar)	Monitors network responses and SSLHandshakeException errors.

Download Link for the Insecure application

[Download App](#)

2. Certificate Pinning implementation guide

Follow these sequential steps to add certificate pinning to your application.

Note: Repeat Steps 2.1 and 2.2 for every target domains your app communicates with to extract their public keys.

2.1 — Capture the server's certificate chain

Open a TLS connection using OpenSSL to dump the full certificate chain and intermediates for each target domain:

```
openssl s_client -connect api.github.com:443 -servername api.github.com -showcerts
```

2.2 — Compute the SHA-256 SPKI Pin

Extract the public key and generate the base64-encoded SHA-256 Subject Public Key Info (SPKI) hash for each certificate:

```
openssl x509 -in leaf.pem -pubkey -noout \  
| openssl pkey -pubin -outform der \  
| openssl dgst -sha256 -binary \  
| openssl enc -base64
```

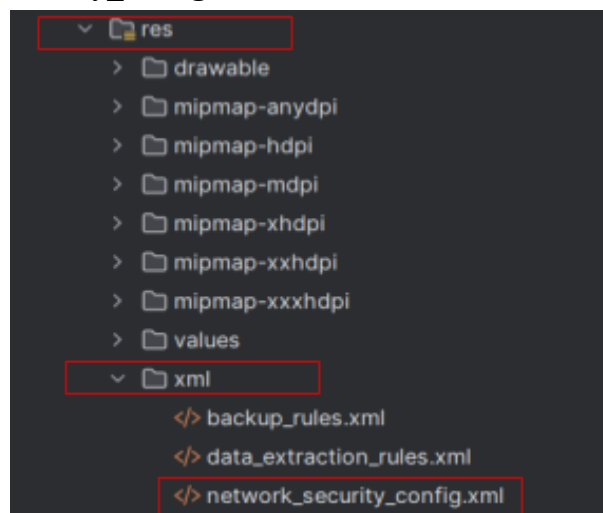
Perform the below steps in Android Studio:

2.3 — Create the Network Security Configuration XML

Create a new configuration file at **app/src/main/res/xml/network_security_config.xml**.

Here are the steps to create the network security configuration file:

1. Switch your Project view from **Android** to **Project** in the top-left corner panel of Android Studio.
2. Navigate to app/src/main/res.
3. Right-click the **res** folder => **New** => **Android Resource Directory**. Set the Resource type to **xml** and click **OK**.
4. Right-click the new **xml** folder => **New** => **XML Resource File**.
5. Name it **network_security_config** and click **OK**.



Define each domain configuration with a **<pin-set>** containing your captured SHA-256 hashes:

Paste the below code in **network_security_config.xml** file :

```
<?xml version="1.0" encoding="utf-8"?>
<network-security-config>
  <domain-config cleartextTrafficPermitted="false">
    <domain includeSubdomains="true">api.github.com</domain>
    <pin-set expiration="2027-09-18">

      <pin digest="SHA-256">S2LUIbq4yUg5w+MYbj5LZOWAZAaeNGJ9rTTc4GjvBQ=</pin>

      <pin digest="SHA-256">ZSagvDzjltLkewXEBuDxIzpW/dpVw1Juvvmd0hhkzdY=</pin>

      <pin digest="SHA-256">sLVjNUaFYfW7n6EtgBeEpjOlcBdNPMrZDRF36iwBdE=</pin>

    </pin-set>
  </domain-config>
</network-security-config>
```

2.4 —Reference the Configuration in AndroidManifest.xml

Declare the **INTERNET** permission and link your network security configuration file inside the **<application>** tag:

```
<uses-permission android:name="android.permission.INTERNET" />
<application
  ...
  android:networkSecurityConfig="@xml/network_security_config">
```

```
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.INTERNET" />

<application
  android:allowBackup="true"
  android:dataExtractionRules="@xml/data_extraction_rules"
  android:fullBackupContent="@xml/backup_rules"
  android:icon="@mipmap/ic_launcher"
  android:label="InsecureCommunication"
  android:roundIcon="@mipmap/ic_launcher_round"
  android:supportsRtl="true"
  android:theme="@style/Theme.InsecureCommunication"
  android:networkSecurityConfig="@xml/network_security_config">
  <activity
```

2.5 — Write Standard Networking Code

No custom pinning logic is needed in Kotlin or OkHttp code. The platform automatically applies the network security config to default clients:

```
val client = OkHttpClient()
val request = Request.Builder().url("https://api.github.com/zen").build()
client.newCall(request).execute()
```

2.6 — Build, Install, and Verify Normal Traffic

Compile and install your debug build using Gradle and ADB commands:

```
./gradlew assembleDebug
adb install -r app/build/outputs/apk/debug/app-debug.apk
adb shell am start -n com.example.securecommunication/.MainActivity
```

Confirm that the app successfully fetches and renders responses from all pinned APIs.

2.7 — Verification : Prove Pin Enforcement

- **Corrupt a Pin:** Temporarily replace one of the real domain pins with an invalid dummy hash in your **network_security_config.xml**

```
<!-- temporarily replace real pins with garbage for one domain -->
<pin
digest="SHA-256">AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAA=</pin>
```

- Rebuild and reinstall the app, then attempt to fetch data from the modified domain.
- **Verify Failure:** Confirm that the corrupted domain fails with an `SSLHandshakeException`: Pin verification failed, while any other correctly pinned domains continue working normally in the same app session.
- **Restore:** Revert the pin back to its correct value and rebuild the app to restore normal functionality.